

Embedded systems I / embedded programming

Introduction

Pascal Sartoretti

v2.0

Introduction: Objectives

Course organisation



- Goal of course
- What is embedded systems ?

Your teacher:

- Scientific cooperator / laboratory manager (23.N320) / SYND order supervisor
- @Hes-so Valais since 1991
- Office 23.N306 / 058 606 8720
- pascal.sartoretti@hevs.ch

Specialist in:

- Embedded programming in C / assembler
- Cooperator in courses ESE
- Microchip PIC / STM32 Cortex
- Microcontroller buses (SPI, I2C, …)
- Field buses (CAN, Modbus, …)

Introduction: What is where ?

Course organisation

All documents are in cyberlearn

- https://cyberlearn.hes-so.ch/
- Connect with your AAI account
- Course name : HES-SO-VS_IP3.11-ProgE
- Course script is English, of course !

Introduction: Course content

CPU chapters:

- Debug Tools
- CPU architecture
- Inputs / outputs
- Interrupts
- Timers
- ADC / DAC
- UART
- I2C
- SPI

Programming chapters:

- C basic
  - Syntax, operators, control
  - Variables, functions
  - Local and global scope
  - Strings, arrays
- C advanced
  - Type casting
  - Structures, unions, bitfields
  - Pointers, function pointers
  - Stack, heap
- XF

Introduction: Course content organisation

Hes·so VALAIS WALLIS

Duration:

- 15 weeks
- 23 sessions of 2 hours (theory/exercises/exams)
  - 7 special ‘C’ programming (incl. XF)
  - 9 special CPU systems
  - 2 preparation exam
  - 2 exams
  - 2 correction exam
  - 1 final exam preparation
- 8 laboratories of 4 hours

Valais excellence

Introduction: PC program vs embedded

What is the difference ?

PC program (all is on your PC)

- Keyboard, display, drivers, …
- Compiler, debugger, …

Embedded (cross development)

- Development on the PC
- Debug on the PC connected to the target
- Program runs on the target
- Target I/O have to be defined

Introduction: Why embedded ?

uControllers are everywhere

- Home
- Transports
- Health
- Communication
-
-

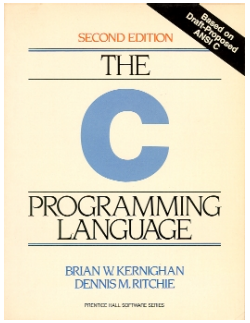
Introduction: References

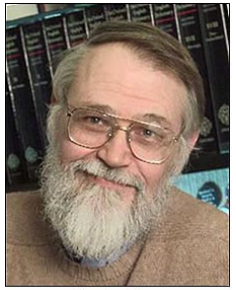
Used hardware

- STM32U5G9-DK2 hardware

C programming reference

- C simple reference by Medard Rieder (in cyberlearn)
- Kernighan & Ritchie




Brian Kernighan


Dennis Ritchie

Introduction: debug tools

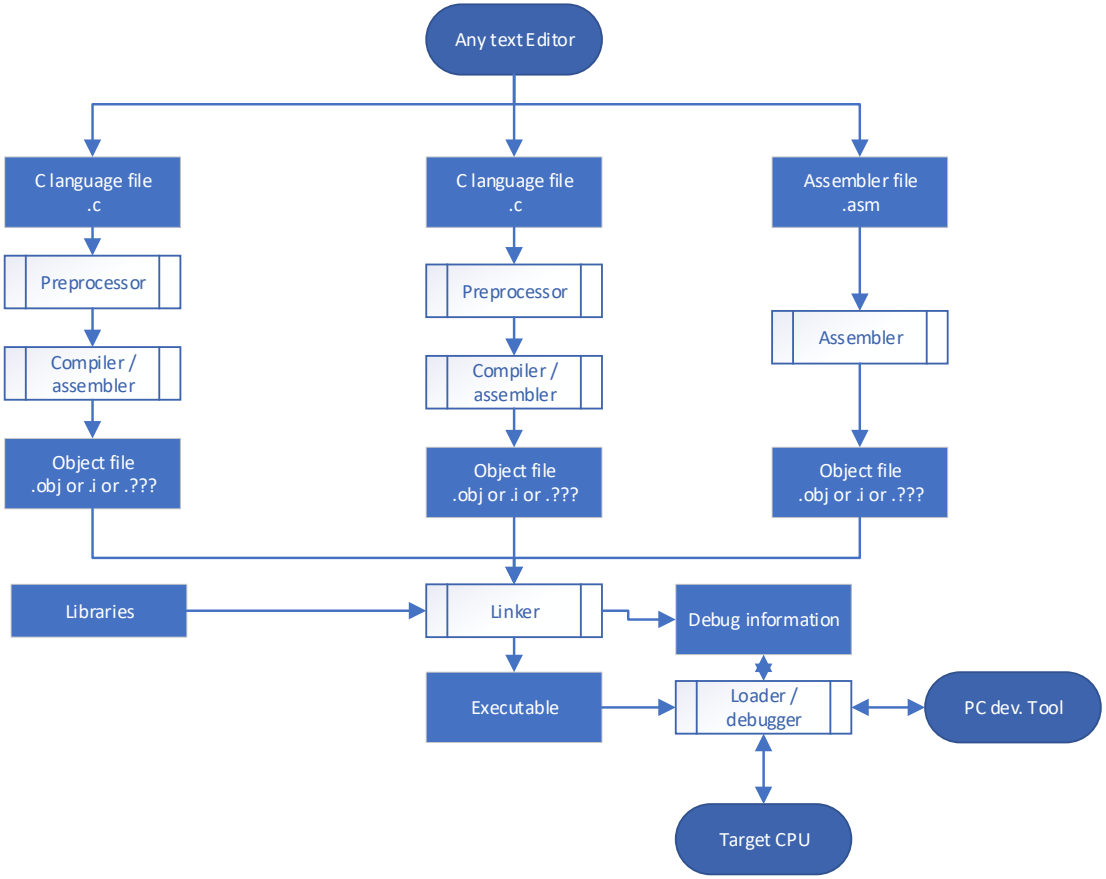
Used PC software



- Visual Studio Code (VSCode)
  - Freeware
  - All STM microcontrollers with extension tool
  - Graphical interface for configuration (on CubeMX)
  - Fast and powerful debug

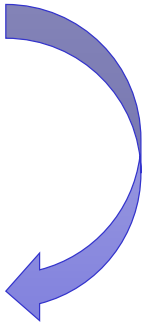
Introduction: From high level down to CPU

- Preprocessor
- Compiler
- Assembler
- Linker
- Loader



Introduction: preprocessor (1/4)

```
#include <stdint.h>
#include "toto.h"
...
uint8_t toto;          // I'm toto
```



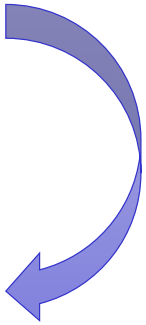
```
#ifndef _STDINT_H
#define _STDINT_H
...
typedef int8_t signed char;
...
#endif _STDINT_H
...
uint8_t toto;
```

Preprocessor 1/4

- Copy/paste #include
- Delete comments

Introduction: preprocessor (2/4)

```
#define MAGIC 255
#define HALFOF(x) x/2
void main(void)
{
  uint8_t toto = MAGIC;
  toto = HALFOF(toto);
}
```



```
void main(void)
{
  uint8_t toto = 255;
  toto = toto/2;
}
```

Preprocessor 2/4

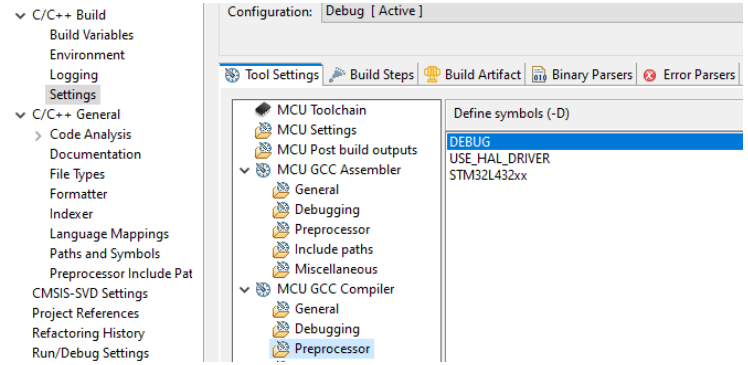
- Interpret #define
- Interpret macros

Introduction: preprocessor (3/4)

```c
void main(void)
{
  #ifdef DEBUG
  printf("Debug on\n");
  #endif
  while(1)
  {}
}
```

```c
void main(void)
{
  while(1)
  {}
}
```

Conditional compilation



STM32CubeIDE

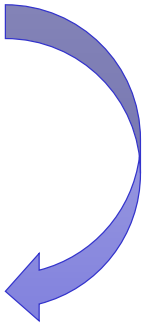
CMakeLists.txt

```
# Add project symbols (macros)
target_compile_definitions(${CMAKE_PROJECT_NAME}
PRIVATE
    # Add user defined symbols
    DEBUG
)
```

VS Code

Introduction: preprocessor (4/4)

```
printf("File is:%s\n",__FILE__);
printf("Line is:%d\n",__LINE__);
printf("Date is:%s\n",__DATE__);
printf("Time is:%s\n",__TIME__);
```



```
printf("File is:%s\n", "main.c");
printf("Line is:%s\n",2);
printf("Date is:%s\n",”13.01.2025”);
printf("Time is:%s\n",”12:42:07”);
```

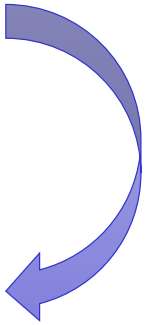
Preprocessor 4/4

- **Predefined macros**

Introduction: compiler

myfile.c

```
uint8_t getDouble(uint8_t number)
{
  return number * 2;
}
```



myfile.s

```
_getDouble:
       lsl     r0, r0, #1
       bx      lr
```

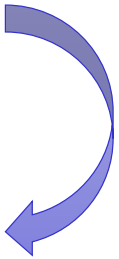
Compiler

- Create assembler code

Introduction: assembler

myfile.s

```
_getDouble:
      lsl     r0, r0, #1
      bx      lr
```



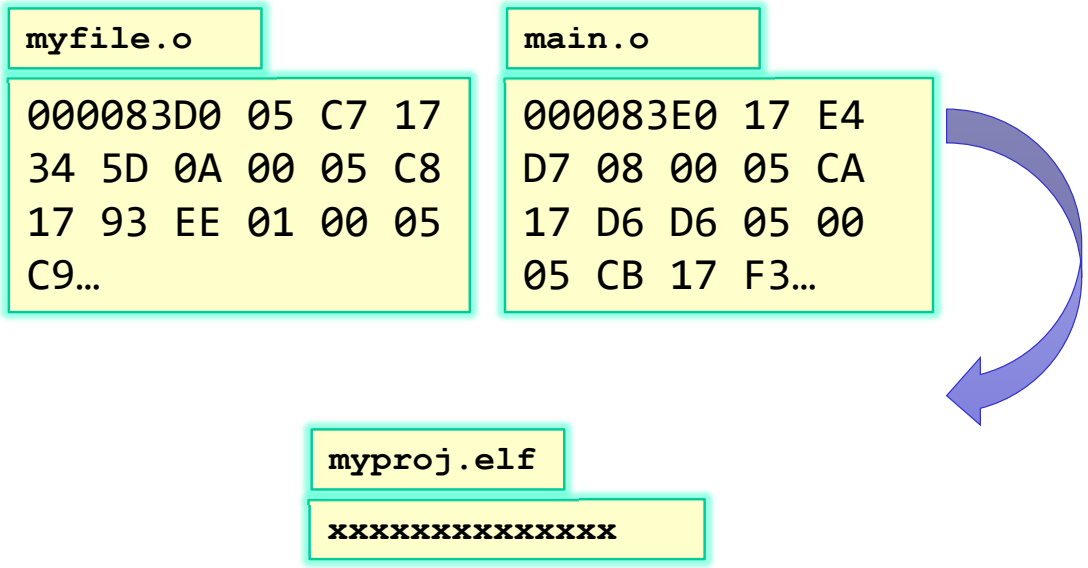
Assembler

- Create machine code

myfile.o

```
000083D0 05 C7 17 34 5D 0A 00 05 C8 17 93 EE 01 00 05 C9 ...4]...........
000083E0 17 E4 D7 08 00 05 CA 17 D6 D6 05 00 05 CB 17 F3 ................
000083F0 60 03 00 05 CC 17 06 B7 0A 00 05 CD 17 78 D2 00 `............x..
```

Introduction: linker



Linker

- **Put object files together**

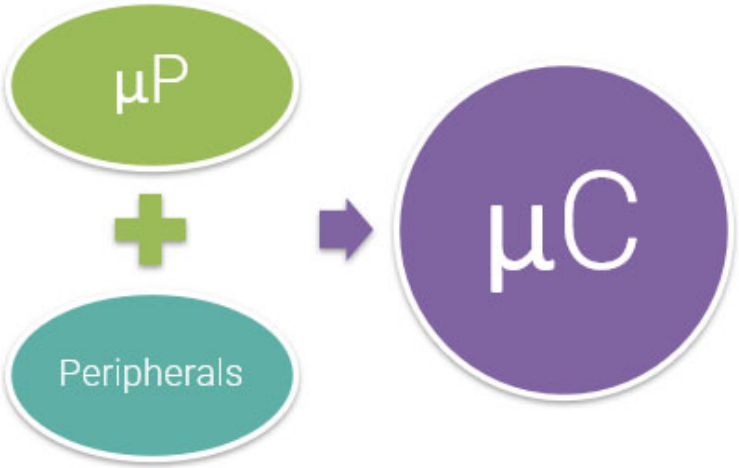
Introduction: CPU vs MCU

Microcontrollers (uC, MCU)

- Contains core, memory, peripherals

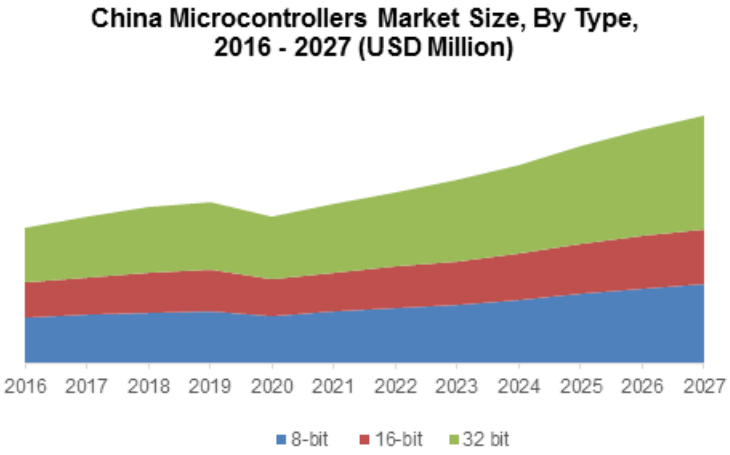
Microprocessors (uP, MPU)

- Contains core
- Only a small memory inside
- Very few peripherals



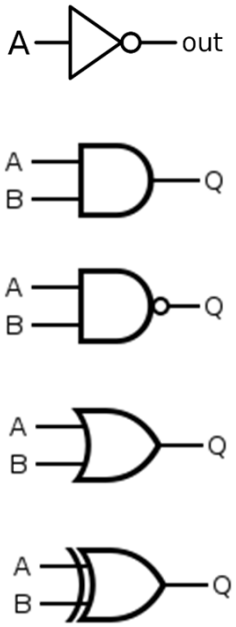
Introduction: CPU vs MCU

INTEL CORE I7	STM32U5G9	PIC18F26K22
No flash	4MB Flash	64kB Flash
12MB CACHE	3MB RAM	4kB RAM
445.-	14.-	3.70
3.6GHz	160MHz	64MHz

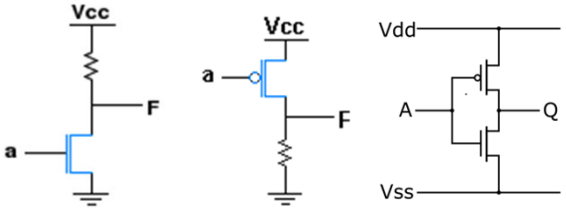


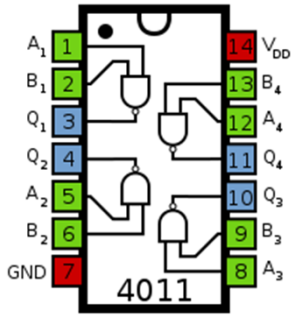
Introduction: digital logic remember

Basic gates



NMOS PMOS Dual

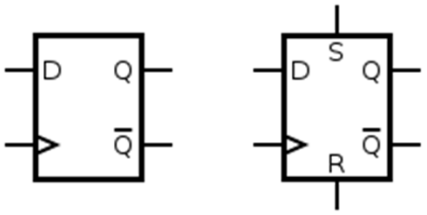




Introduction: digital logic remember

D flip-flop

- Synchronous element
- Can be set and reset
- Used in registers
- ~12 transistor / bit

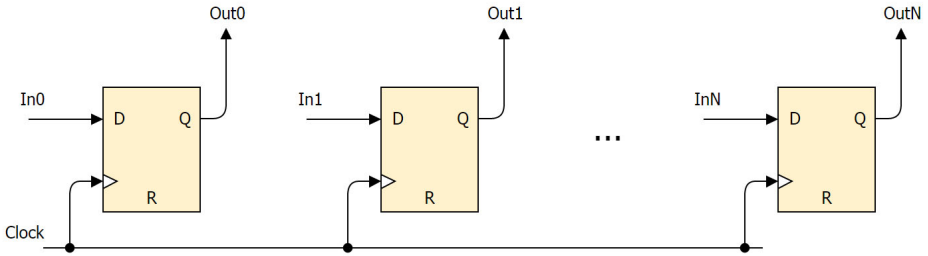


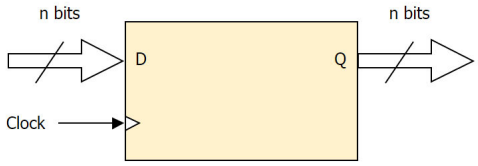
Clock	D	Q_+
Rising edge	0	
Rising edge	1	
Non-rising	X	

Introduction: memory element

n bits register

- n flip-flop in parallel
- Very fast (no latency at all)
- Inside processors, tens of n registers
- Bits numbering: MSB to LSB (left to right)





A n bit register

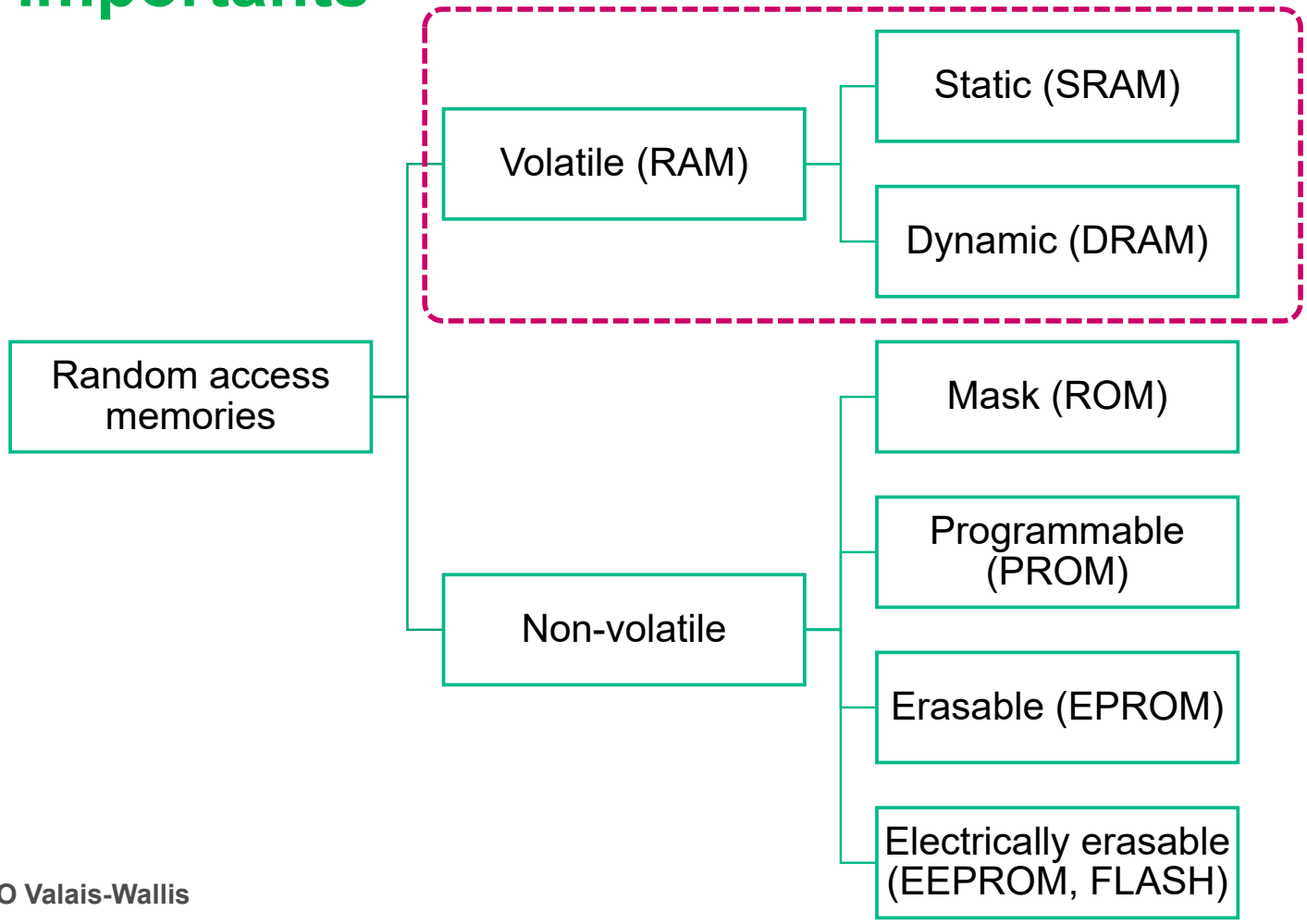
Introduction: memory element

Multiples in computer science

- A bit : single binary information
- A Byte : 8 bits
- A nibble : 4 bits
- A word :
- A kiloBytes :
- A megaBytes :
- A gigaBytes :
- A teraBytes :

Introduction: memory arrays

The most importants



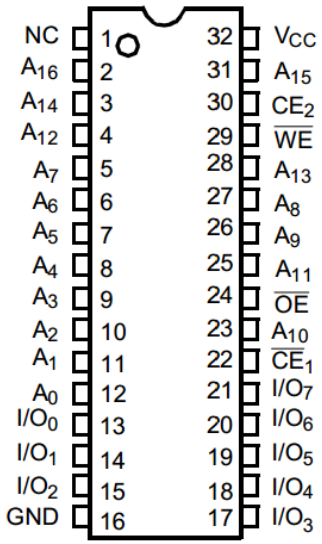
Introduction: static memory

- Simple to use
- Address bits :
- Data bits :
- Control bits : nWE, nOE, nCE

What is the size of this SRAM ?
One many cycles to write 32 bits ?

Pin Configurations

Top View
SOIC



62128V-2

Introduction: dynamic memory

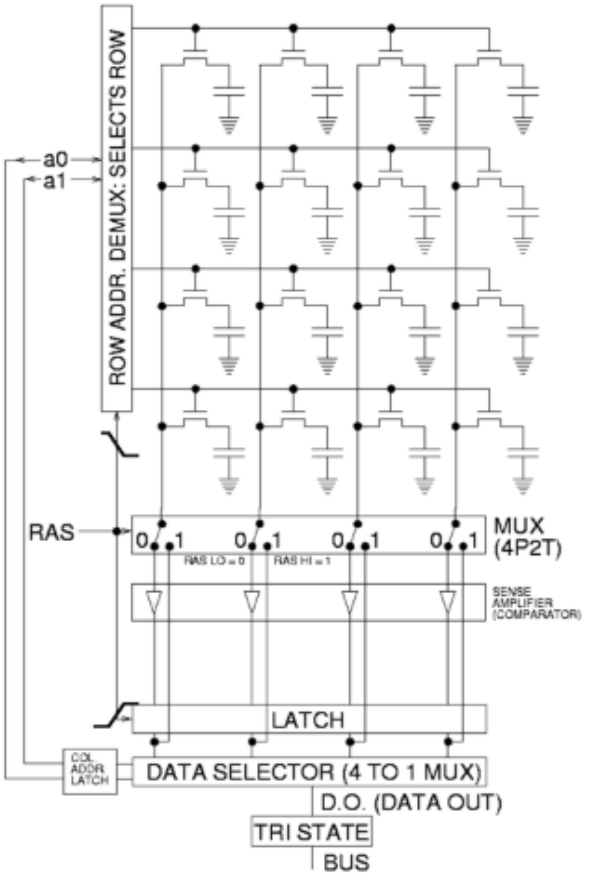
Very Different from static memory

- 1 bit of memory = 1 transistor
- Very fast
- Very cheap

Various flavours

- DRAM, DDR, DDR2, DDR3, …

Without refresh, data is keept max. 64 ms

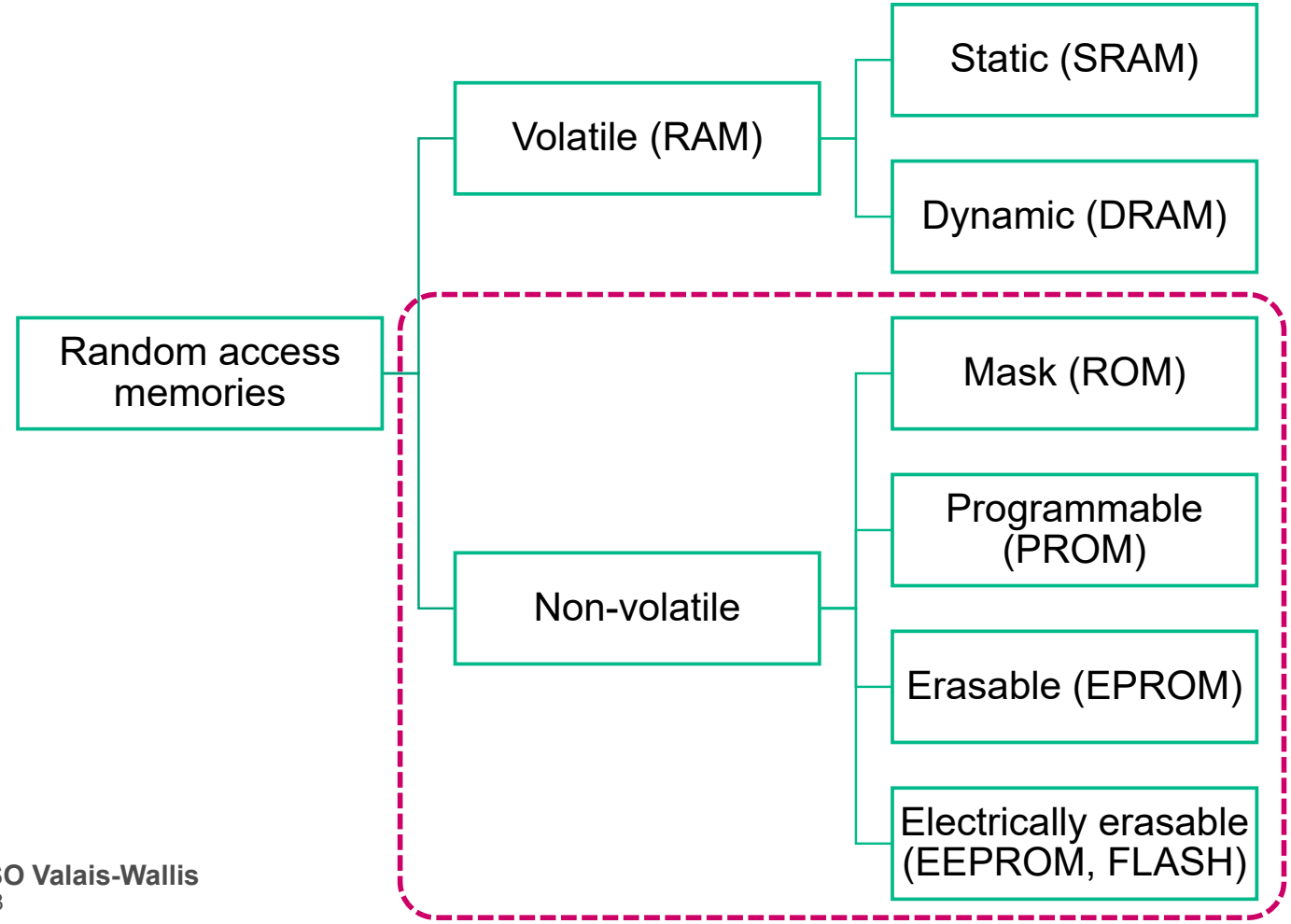


Introduction: static vs dynamic memories

SRAM	DRAM
It stores information as long as the power is supplied.	It stores information as long as the power is supplied or a few milliseconds when power is switched off.
Transistors are used to store information in SRAM.	Capacitors are used to store data in DRAM.
Capacitors are not used hence no refreshing is required.	To store information for a longer time, contents of the capacitor needs to be refreshed periodically.
SRAM is faster compared to DRAM.	DRAM provides slow access speeds.
It does not have a refreshing unit.	It has a refreshing unit.
These are expensive.	These are cheaper.
SRAMs are low-density devices.	DRAMs are high-density devices.
In this bits are stored in voltage form.	In this bits are stored in the form of electric energy.
These are used in cache memories.	These are used in main memories.
Uses less power and generates less heat.	Consumes more power and generates more heat.

Introduction: memory arrays

The most importants



Introduction: non volatile memories

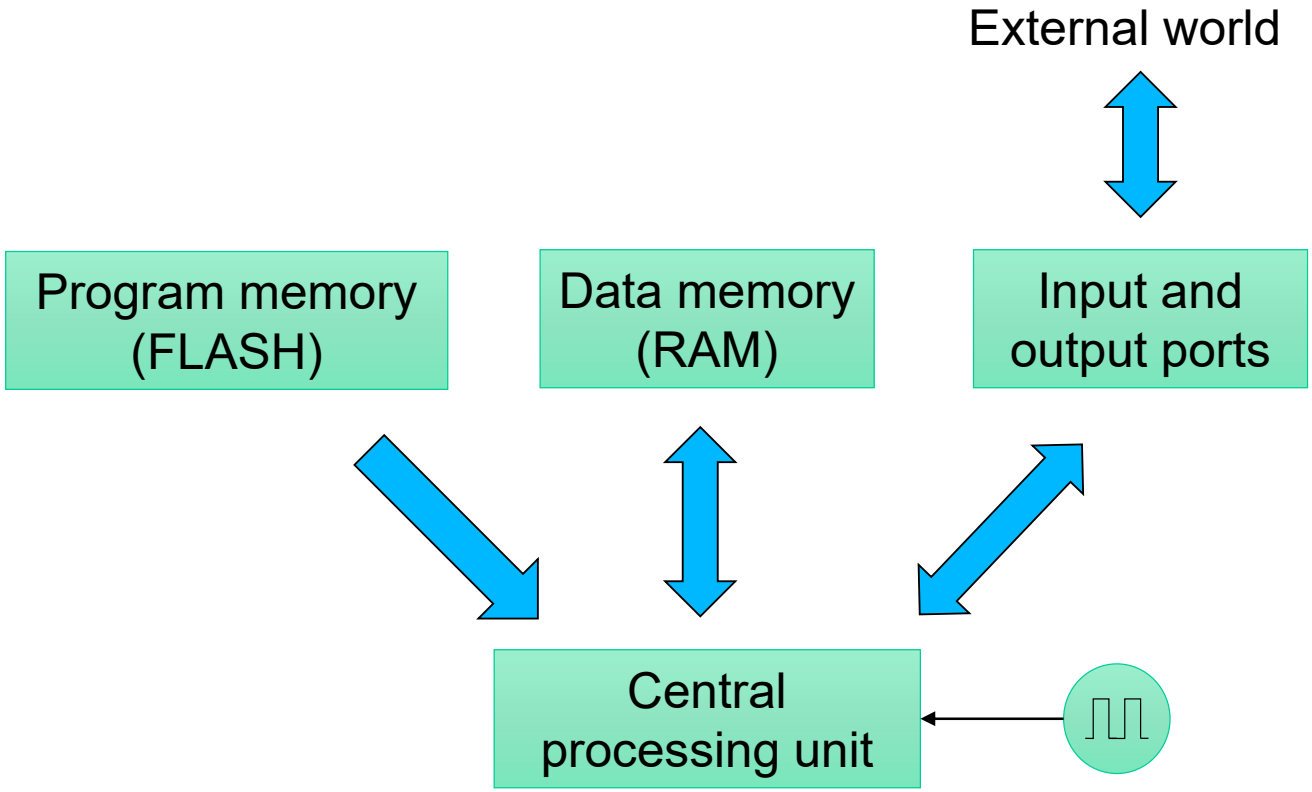
- Content is kept when no power applied
- Slower than SRAM
- Exist in different technologies
  - ROM, EPROM, EEPROM, Flash
- Flash are nonsymmetrical (read != write)
  - Read any byte
  - Erase sector (special command)
  - Write bytes (special command)
  - Erase chip (special command)

Introduction: magnetic memories

- Different versions (MRAM, FRAM)
- Relative new technology (~2025)
  - Everspin technology
- Low densities (max. 32Mbits)
- Expensive (5 $ for 32kB)

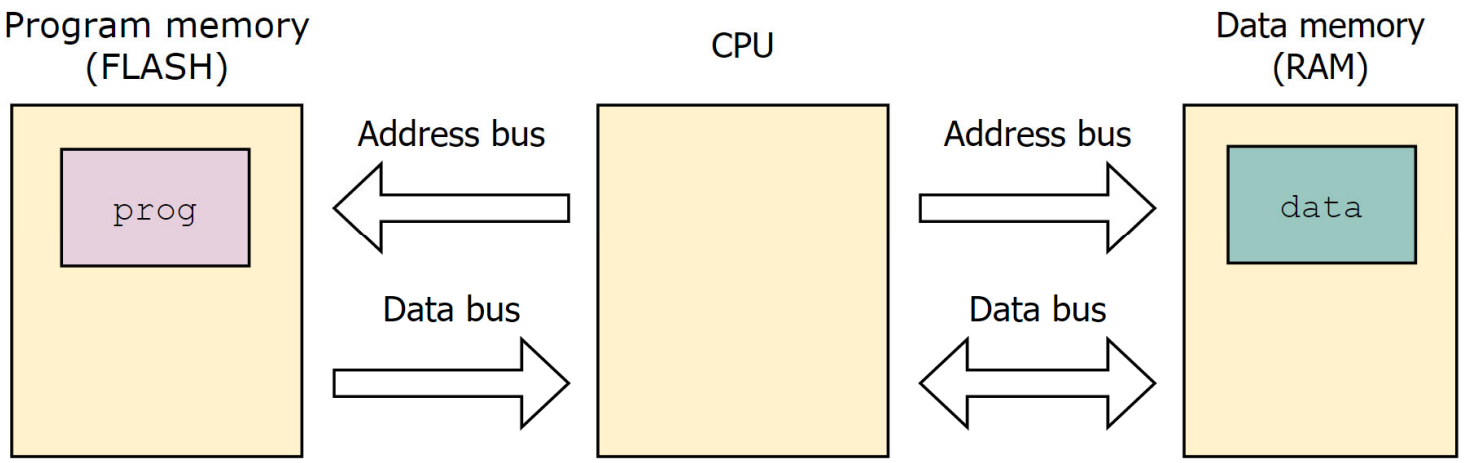
Architecture : uC components

What is in a uC ?



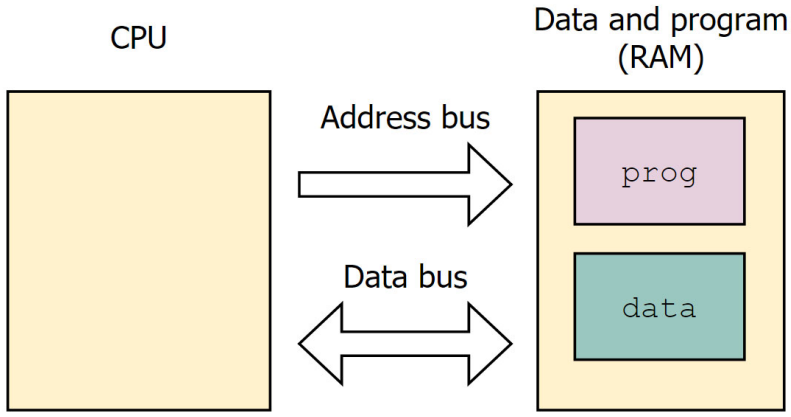
Architecture : Harvard organization

- One bus for code (program)
- One another bus for variables (data)



Architecture : Von Neumann organization

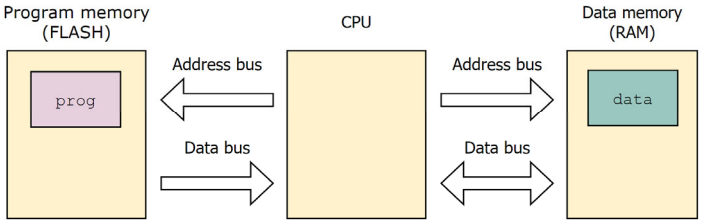
- **Same bus for code (program) and variables (data)**



Architecture : Harvard vs Von Neumann

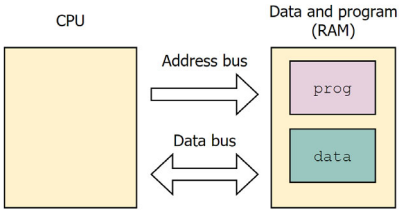
Harvard

- Separate memories for data and code
- Two buses between CPU and memory



Von neumann

- Same memory for data and code
- One single bus between CPU and memory

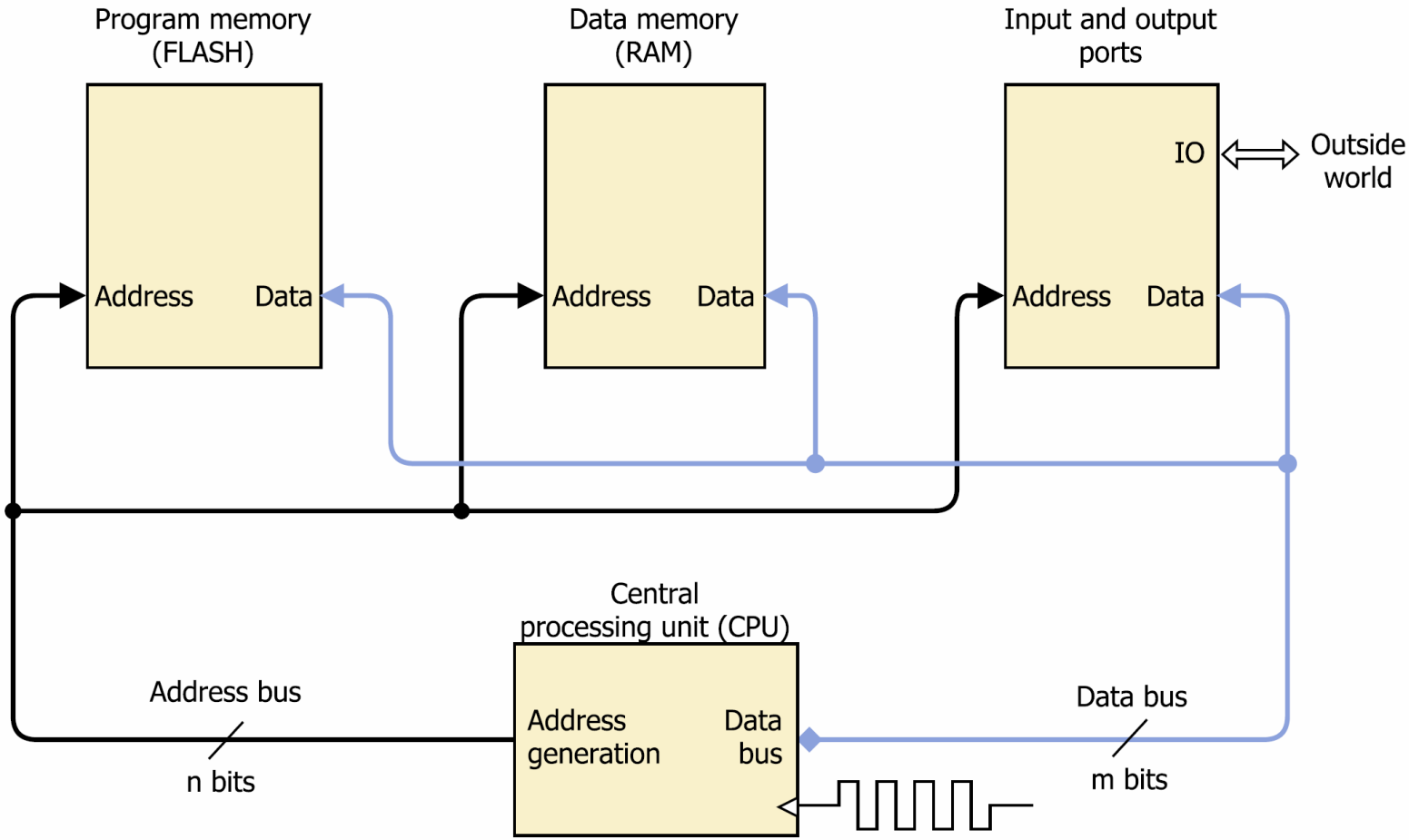


Architecture : Harvard vs Von Neumann

VON NEUMANN ARCHITECTURE	HARVARD ARCHITECTURE
It is ancient computer architecture based on stored program computer concept.	It is modern computer architecture based on Harvard Mark I relay based model.
Same physical memory address is used for instructions and data.	Separate physical memory address is used for instructions and data.
There is common bus for data and instruction transfer.	Separate buses are used for transferring data and instruction.
Two clock cycles are required to execute single instruction.	An instruction is executed in a single cycle.
It is cheaper in cost.	It is costly than van neumann architecture.
CPU can not access instructions and read/write at the same time.	CPU can access instructions and read/write at the same time.
It is used in personal computers and small computers.	It is used in micro controllers and signal processing.

Source https://www.geeksforgeeks.org/difference-between-von-neumann-and-harvard-architecture/

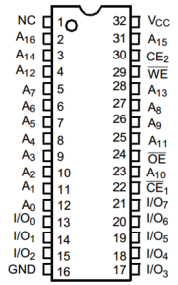
Architecture : Components

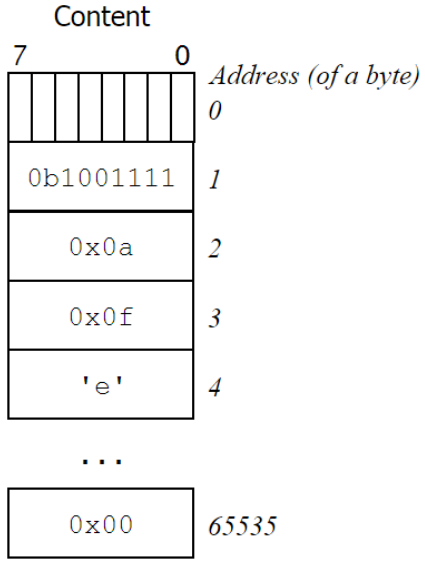


Architecture : Memory organization

RAM sdresses are organized by **bytes**

- Each address line correspond to a single byte
- For variables of a size bigger than a byte, consecutive bytes are used to store this information
- Caution with bytes order for variables bigger than one byte !

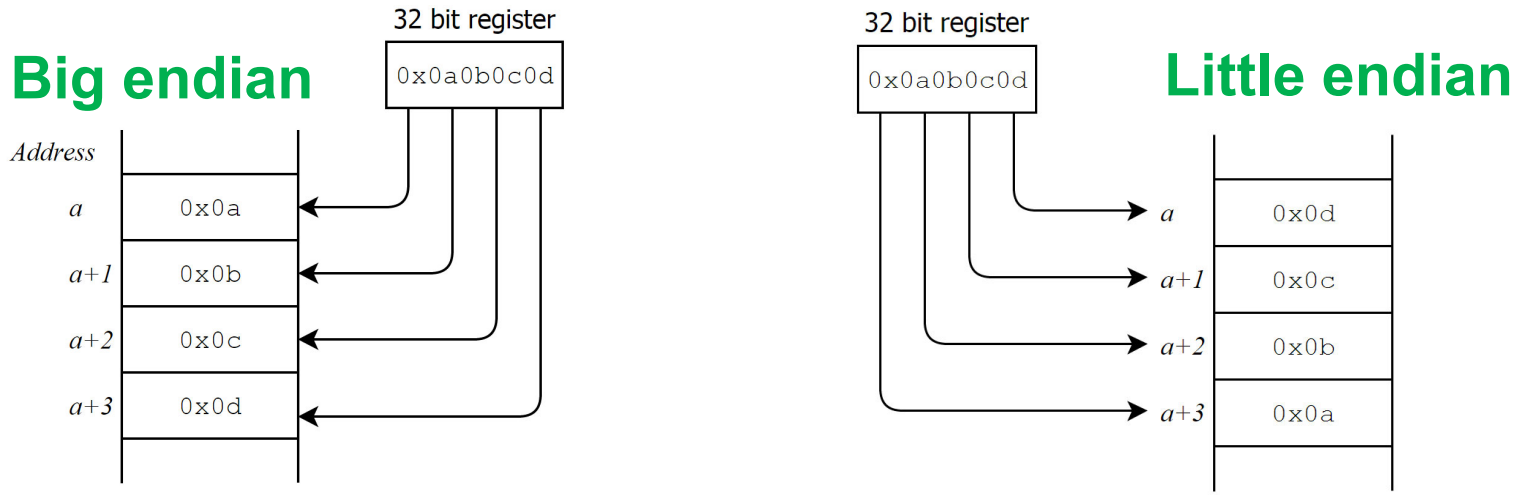




Architecture : Endianness

We want to store a 32 bits value containing `0x0A0B0C0D`

- **Two solutions available**
- **No endianness in registers**

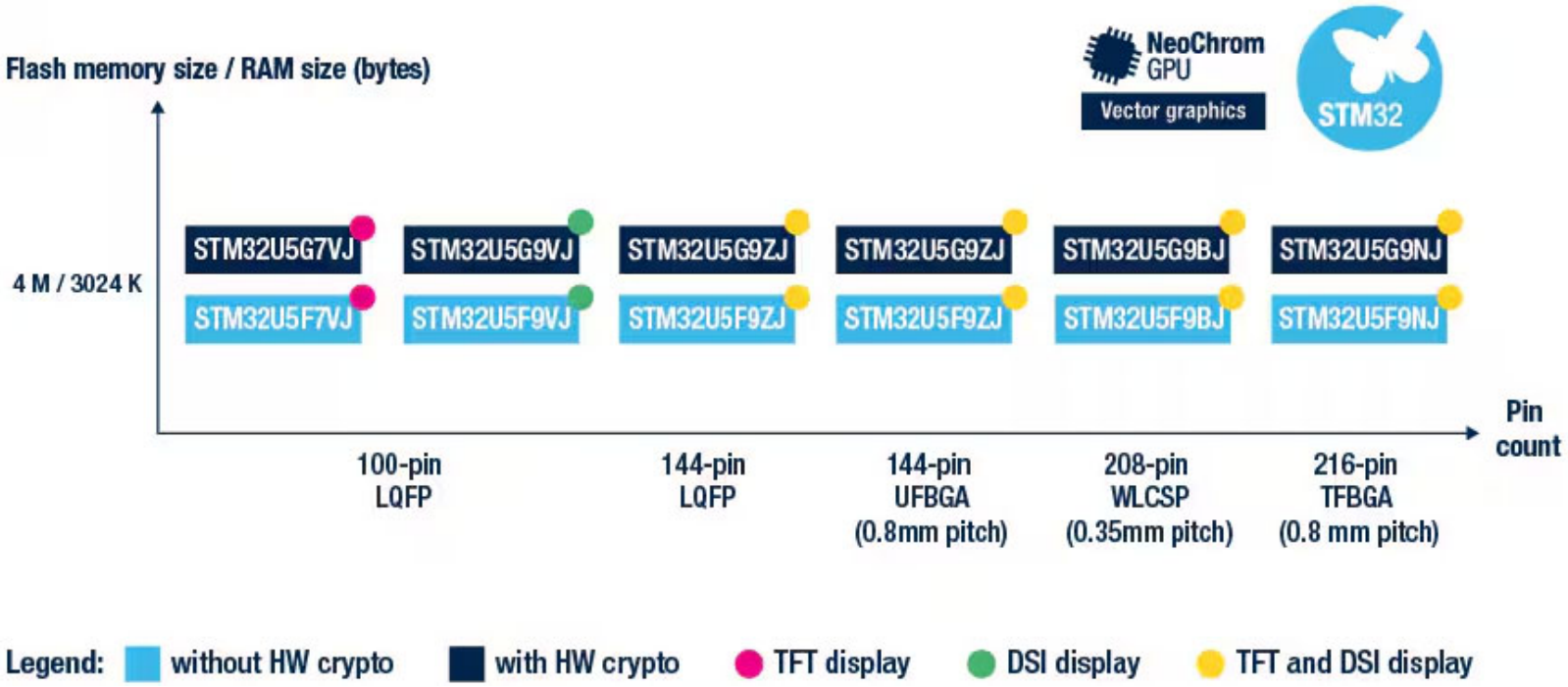


Architecture : STM32U5G9

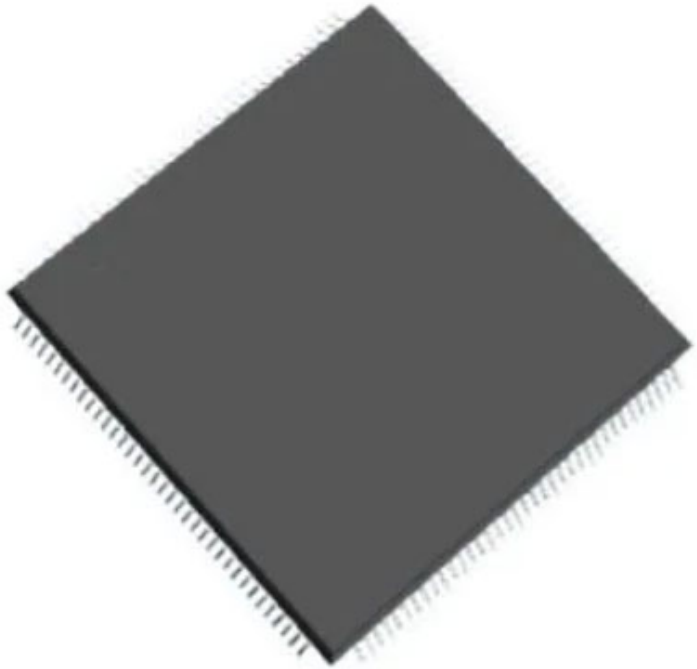
Cortex-M33 architecture (2016)

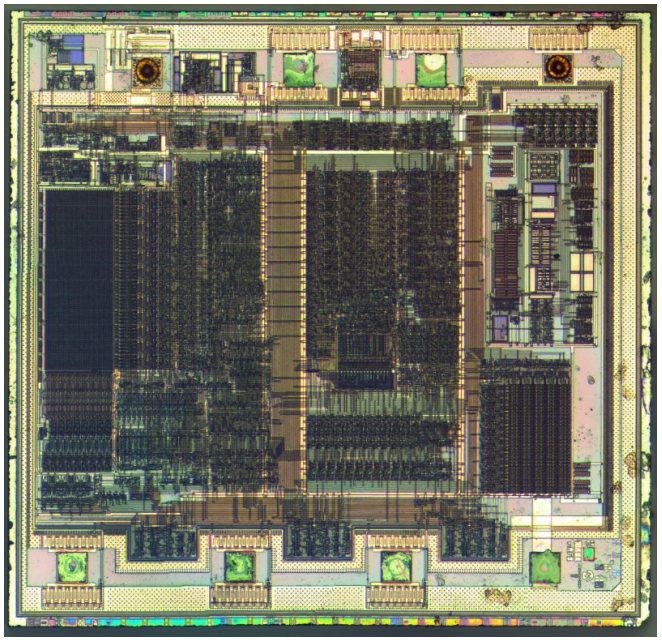
- Data bus width = 32 bits
- ALU is 32 bits
- Memory access is 32 bits

Architecture : STM32U5G9 family



Architecture : STM32U5G9 looks like …





Architecture : Instruction Set Architecture

The ISA defines:

- The system's state (registers, memory, PC, …)
- The instructions CPU can execute
- The effects of theses instructions on the system state

Two main standards

- RISC (Reduced Instruction Set Computer)
- CISC (Complex Instruction Set Computer)

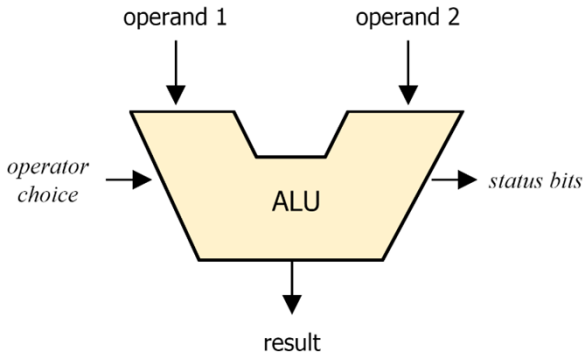
STM32U5G9 is a Cortex-M33 RISK core

Architecture : The STM32U5G9 ISA (M33)

- More than 200 instructions
- 32 bits data path
- Von Neuman architecture
  - 3 MBytes data SRAM (for variables)
  - 4 MBytes program Flash (for program)
- 32 bits instructions (or 16)
- 16 working register (+48)

Architecture : Important registers

- 13 standard working registers (r0-r12)
- The PC (Program Counter)
- The SP (Stack Pointer)
- The LR (Location Register)
- The XPSR (status and more)



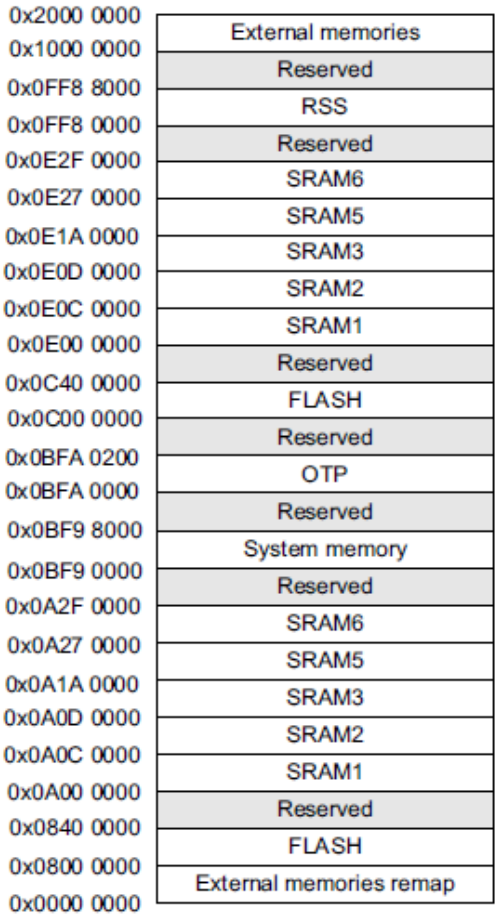
ALU stands for mathematical and logical operations (+, -, NOT, OR, AND, shift, rotate, …)

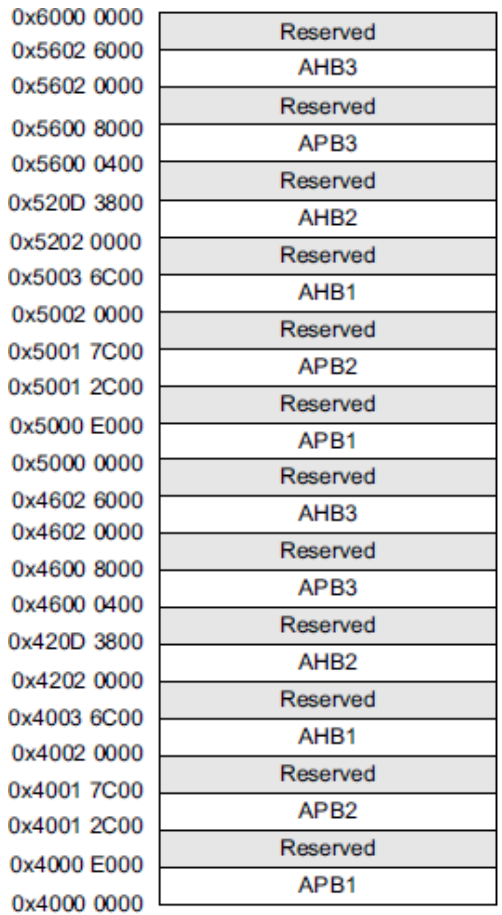
Architecture : Memory map STM32U5G9

From programmer's perspective, everything we work with has an address

Those addresses can correspond to:

- Memory locations
- Peripherals
- I/O ports





Architecture : Complete block diagram

